

P0929R1: Checking for abstract class types

Introduction

Subclause 13.4 [class.abstract] paragraph 3 states:

An abstract class shall not be used as a parameter type, as a function return type, or as the type of an explicit conversion. Pointers and references to an abstract class can be declared.

This is troublesome, because it requires checking a property only known once a type is complete at a position (declaring a function) where the rest of the language permits incomplete types to appear.

In practice, this introduces spooky "backward-in-time" errors:

```
struct S;  
S f();    // #1, ok  
  
// lots of code  
  
struct S { virtual void f() = 0; }; // makes #1 retroactively ill-formed
```

Changes

Relative to P0929R0:

- Treat arrays of abstract class type similarly.
- Update wording.

Discussion

Declarations should add information to the state of a program, not retroactively make existing constructs that have been parsed successfully ill-formed.

Beyond the ability to declare functions with incomplete return and parameter types, the standard also allows for the return type to be inspected using `decltype(f())` even if it is incomplete. This ability might be used in template metaprogramming, and it should not break if one of the involved types happens to be abstract.

There are two approaches to solving this issue:

- State that a return type or a parameter type is ill-formed if it is complete and an abstract class type at the point of declaration of the function type.
- Never check declarations involving function types whether they involve a return type or parameter of abstract class type. Instead, postpone the check to the definition and the call, where actual objects need to be created, and where the existing text already requires completeness of the types involved.

The first option seems bizarre in that token-by-token identical redeclarations may become ill-formed:

```

struct S;
S f();      // ok, abstract-ness not checked here
struct S { virtual void f() = 0; };
S f();      // error: S is abstract

```

Therefore, the proposed wording chooses the second option.

It is recommended to treat this as a Defect Report against earlier versions of C++.

The suggested feature test macro is `__cpp_abstract_class_checking`.

Wording changes

Change in 6.1 [basic.def] paragraph 15:

~~A program is ill-formed if~~ In the definition of ~~any~~ **an** object ~~gives the~~, **the type of that object shall not be** an incomplete type (6.7 [basic.types]), **an abstract class type (13.4 [class.abstract]), or a (possibly multi-dimensional) array thereof.**

Change in 8.2.1 [basic.lval] paragraph 9:

Unless otherwise indicated (~~8.5.1.2 [expr.call]~~ **10.1.7.2 [dcl.type.simple]**), a prvalue shall always have complete type or the void type; **if it has a class type or (possibly multi-dimensional) array of class type, that class shall not be an abstract class (13.4 [class.abstract]).**

Change in 8.5.1.2 [expr.call] paragraph 4:

... When a function is called, ~~the parameters that have object type shall have completely defined object type~~ **the type of any parameter shall not be an incomplete or abstract class type.** [Note: this still allows a parameter to be a pointer or reference to an incomplete class type. However, it prevents a passed-by-value parameter to have an incomplete class type. -- end note] ...

Change in 11.3.4 [dcl.array] paragraph 1:

... T is called the *array element type*; this type shall not be a reference type, cv void, **or** a function type ~~or an abstract class type~~ ...

Change in 11.3.5 [dcl.fct] paragraph 12:

Types shall not be defined in return or parameter types. The type of a parameter or the return type for a function definition shall not be an incomplete **or abstract** (possibly cv-qualified) class type in the context of the function definition unless the function is deleted (11.4.3).

Editorial note: I suggest to move the statement about function definitions to 11.4 [dcl.fct.def].)

Change in 12.2 [class.mem] paragraph 14:

~~Non-static~~ **The type of a non-static** data ~~members~~ **member** shall not ~~have~~ **be an** incomplete ~~types~~ **type (6.7 [basic.types]), an abstract class type (13.4 [class.abstract]), or a (possibly multi-dimensional) array thereof. [Note:** In particular, a class C ~~shall not~~ **cannot** contain a non-static member of class C, but it can contain a pointer or reference to an object of class C. **-- end note]**

Change in 13.4 [class.abstract] paragraph 2:

An *abstract class* is a class that can be used only as a base class of some other class; no objects of an abstract class can be created except as subobjects of a class derived from it. A class is abstract if it has at least one *pure virtual function*. [Note: Such a function might be inherited: see below. -- end note] A virtual function is specified **as a pure virtual function** by using a *pure-specifier* (12.2) in the function declaration in the class definition. [Note: Such a function might be inherited: see below. -- end note] A class is an *abstract class* if it has at least one pure virtual function. [Note: An abstract class can be used only as a base class of some other class; no objects of an abstract class can be created except as subobjects of a class derived from it (6.1 [basic.def], 12.2 [class.mem]). -- end note] A pure virtual function need be defined only if called with, or as if with (15.4), the *qualified-id* syntax (8.4).

Remove 13.4 [class.abstract] paragraph 3:

An abstract class shall not be used as a parameter type, as a function return type, or as the type of an explicit conversion. Pointers and references to an abstract class can be declared. [Example:

```
shape x; // error: object of abstract class
shape* p; // OK
shape f(); // error
void g(shape); // error
shape& h(shape&); // OK
```

-- end example]

Remove 17.9.2 [temp.deduct] paragraph 11 bullet 11:

- ...
- Attempting to create a function type in which a parameter type or the return type is an abstract class type (13.4 [class.abstract]).